GUÍA OFICIAL DE CALIDAD & TDD

# Catálogo Maestro de Pruebas de Software

Pruebas Unitarias, de Integración y End-to-End (Explicadas a Detalle y Sin Resolver)


**EQUIPO DE DESARROLLO**
Cristian Beltrán & Compañero


**METODOLOGÍA DE PRUEBAS**
Test-Driven Development (TDD)

**ENTORNO DE PRUEBAS**
SQLite en Memoria (BaseTestCase)

## 1. ¿Qué son las Pruebas de Software? (Para Principiantes) 🧸

### 🔍 La Fábrica de Juguetes y el Inspector con Lupa

Imaginen que estamos construyendo un carro de carreras con piezas de LEGO. ¿Verdad que no lo armamos todo con los ojos cerrados para ver si rueda al final? ¡No! Probamos pieza por pieza:

**1. Pruebas Unitarias (Probar una sola ruedita):** Revisamos con una lupa si una sola pieza funciona bien por sí sola, sin importar el resto del carro. Por ejemplo: ¿la función para sumar horas sabe que después de las 8:00 AM vienen las 4:00 PM? ¿la función de contraseñas convierte la palabra secreta en código encriptado?

**2. Pruebas de Integración (Juntar la rueda con el motor):** Probamos si dos piezas distintas encajan y se entienden bien cuando se tocan. Por ejemplo: cuando el doctor crea una receta médica en el servicio, ¿la base de datos realmente guarda los datos en su tabla sin rechinar?

**3. Pruebas End-to-End o E2E (El paseo completo del carro en la pista):** Probamos el recorrido completo que hace una persona real: desde que entra a la aplicación, se registra, inicia sesión, recibe su llave mágica (JWT), da de alta un medicamento y revisa su calendario de tomas.

### 📐 El Patrón Universal de Toda Prueba: AAA (Arrange - Act - Assert)

Cada una de las pruebas que verán a continuación sigue rigurosamente tres pasos en su interior:

- **1. Preparar (Arrange):** Se definen los datos de entrada (ejemplo: un diccionario con nombre, correo y clave).
- **2. Actuar (Act):** Se llama a la función, método o ruta HTTP que se quiere evaluar.
- **3. Verificar (Assert):** Se comprueba matemáticamente que el resultado obtenido sea exactamente igual al resultado esperado (ejemplo: `self.assertEqual(codigo, 201)`).

## 2. Diagrama Relacional de Referencia (Base de Datos medicent_v3) 🖼️

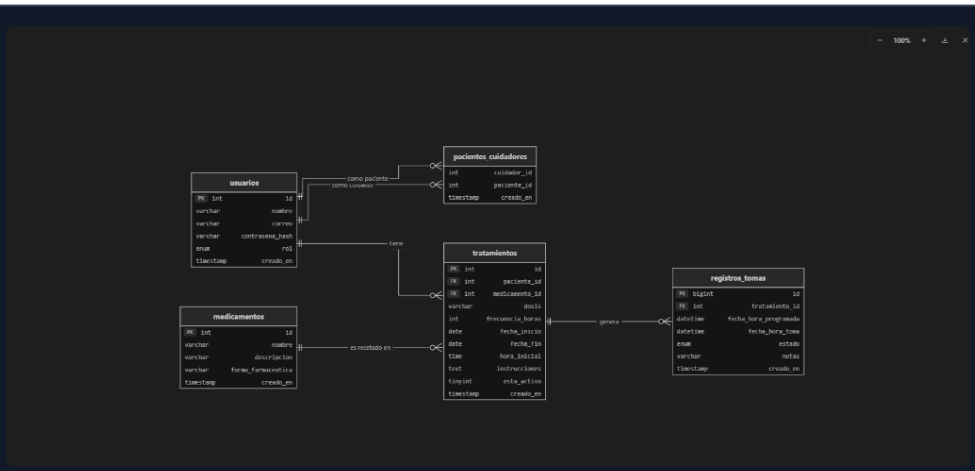



*Figura 1: Esquema de Entidad-Relación de Medicent Backend V3 (usuarios, pacientes_cuidadores, medicamentos, tratamientos, registros_tomas).*

## 3. Catálogo de Pruebas Unitarias (Nivel 1 - Componentes Aislados) 🧪

Se ejecutan en `tests/test_modelos.py` y `tests/test_servicios.py`. Evalúan modelos y algoritmos puros en memoria.

### 1. `test_creacion_modelo_usuario_campos_validos`

UNITARIA Cristian

**Objetivo:** Verificar que la clase `Usuario` se instancie correctamente con nombre, correo, hash de contraseña y rol asignado.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Datos válidos de usuario con rol 'cuidador'. | Instanciar `u = Usuario(...)` y persistir en la sesión. | `u.id` asignado, `u.rol == 'cuidador'`, `to_dict()` no expone contraseña en texto plano. |

### 2. `test_validacion_unicidad_correo_usuario`

UNITARIA Cristian

**Objetivo:** Asegurar que el sistema impida guardar dos cuentas con la misma dirección de correo electrónico.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Guardar un usuario con correo 'test@medicent.com'. | Intentar guardar un segundo usuario con el mismo correo. | La base de datos arroja `IntegrityError` y la sesión hace rollback seguro. |

### 3. `test_relacion_muchos_a_muchos_paciente_cuidador`

UNITARIA Cristian

**Objetivo:** Validar que la tabla asociativa `pacientes_cuidadores` vincule correctamente a un cuidador con múltiples pacientes.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| 1 usuario cuidador y 2 usuarios pacientes en la BD. | Asociar ambos pacientes a la lista de pacientes del cuidador. | La relación refleja 2 pacientes vinculados al cuidador y la clave foránea es consistente. |

### 4. `test_hashing_y_verificacion_password_bcrypt`

UNITARIA Cristian

**Objetivo:** Verificar que `auth_service.py` encripte contraseñas con sal y valide hashes correctamente.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Contraseña en texto plano: 'ClaveSegura2026!'. | Generar hash y luego verificar con la clave correcta y con una errónea. | El hash es diferente al texto plano; valida True con la correcta y False con la incorrecta. |

### 5. `test_creacion_modelo_medicamento`

UNITARIA Tu Parte

**Objetivo:** Validar la creación de una entidad en el catálogo farmacológico con su forma farmacéutica.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Nombre: 'Paracetamol', Forma: 'Comprimido 500mg'. | Crear y guardar instancia de `Medicamento` en SQLite. | El registro existe en BD con timestamp `creado_en` y ID asignado. |

### 6. `test_algoritmo_calculo_tomas_intervalo_regular`

UNITARIA Tu Parte

**Objetivo:** Comprobar la matemática del motor de tomas: para un tratamiento de 2 días cada 8 horas debe generar exactamente 6 tomas.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Inicio: 2026-11-01 08:00, Fin: 2026-11-02 23:59, Frecuencia: 8 horas. | Invocar la función `calcular_horarios_tomas(...)`. | Lista con exactamente 6 timestamps con intervalos exactos de 8 horas: 08:00, 16:00, 00:00, etc. |

### 7. `test_tomas_generadas_estado_inicial_pendiente`

UNITARIA Tu Parte

**Objetivo:** Confirmar que toda toma recién generada por el motor clínico posea el estado 'pendiente' y hora_toma nula.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Generar las tomas para un tratamiento nuevo. | Inspeccionar los atributos de cada registro de toma creado. | `toma.estado == 'pendiente'` y `toma.fecha_hora_toma is None` en el 100% de tomas. |

### 8. `test_transicion_estado_toma_valida`

UNITARIA Tu Parte

**Objetivo:** Verificar que `toma_service.py` permita cambiar el estado a 'tomada', 'omitida' o 'retrasada' y asigne fecha actual.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
| --- | --- | --- |
| Una toma en estado 'pendiente'. | Llamar a `marcar_toma(id, nuevo_estado='tomada')`. | `toma.estado == 'tomada'` y `toma.fecha_hora_toma` contiene timestamp válido. |

## 4. Catálogo de Pruebas de Integración (Nivel 2 - Conexión de Capas) 🧩

Se ejecutan en `tests/test_servicios.py` y middlewares. Evalúan la interacción entre dos o más módulos.

### 9. `test_integracion_servicio_autenticacion_y_jwt`

INTEGRACIÓN Cristian

**Objetivo:** Validar el flujo de autenticación: buscar usuario en BD, verificar contraseña hasheada y codificar el token JWT con sus claims.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Usuario registrado en BD SQLite de prueba. | Llamar a `auth_service.iniciar_sesion(correo, password)`. | Retorna dict con token JWT decodificable que contiene el `id` y `rol` correctos. |

### 10. `test_integracion_middleware_rol_requerido_rechazo`

INTEGRACIÓN Cristian

**Objetivo:** Comprobar que el decorador `@rol_requerido('cuidador')` bloquee la ejecución si el token pertenece a un 'paciente'.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Generar token JWT con claim `rol = 'paciente'`. | Invocar función protegida decorada con `@rol_requerido('cuidador')`. | Se aborta la petición con código 403 Forbidden y mensaje de permiso denegado. |

### 11. `test_integracion_creacion_tratamiento_con_persistencia_en_cascada`

INTEGRACIÓN Tu Parte

**Objetivo:** Probar que al guardar un tratamiento en `tratamiento_service.py` se guarden en cascada todas las tomas en la BD.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Paciente y Medicamento existentes en BD SQLite. | Ejecutar `crear_tratamiento(...)` por 3 días cada 12 horas. | El tratamiento se guarda con su ID y en la tabla `registros_tomas` existen exactamente 6 registros vinculados. |

### 12. `test_integracion_transicion_estado_invalido_rechazada`

INTEGRACIÓN Tu Parte

**Objetivo:** Asegurar que si se intenta colocar un estado no permitido (ej. 'cancelado' o 'destruida'), el servicio lance excepción de dominio.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Una toma existente en base de datos. | Invocar `toma_service.marcar_toma(id, estado='invalido')`. | Lanza `ValueError` o excepción personalizada y la BD permanece inmutable. |

## 5. Catálogo de Pruebas End-to-End (E2E) (Nivel 3 - Flujo de API Completo) 🌐

Se ejecutan en `tests/test_rutas_api.py` utilizando el cliente de pruebas HTTP de Flask (`self.client`). Representan las pruebas conjuntas.

**13. `test_e2e_healthcheck_servidor_activo`** — END-TO-END — Ambos

**Objetivo:** Verificar que la ruta pública de verificación de salud de la API responda con código 200 y confirmación JSON.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Cliente de pruebas levantado. | `self.client.get('/api/salud')`. | Status HTTP 200 y JSON `{"estado": "activo"}` o equivalente. |

**14. `test_e2e_flujo_completo_registro_y_login`** — END-TO-END — Ambos

**Objetivo:** Realizar el ciclo de usuario completo: registrar una cuenta nueva y luego hacer login para obtener un token JWT válido.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Payload con nombre, correo nuevo y contraseña. | POST a `/api/auth/registro` y luego POST a `/api/auth/login`. | Registro retorna HTTP 201; Login retorna HTTP 200 y entrega llave `token_acceso`. |

**15. `test_e2e_login_con_credenciales_invalidas_rechazado`** — END-TO-END — Ambos

**Objetivo:** Validar que una petición de login con contraseña incorrecta sea rechazada de forma segura sin revelar información de más.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Usuario registrado con clave 'Correcta123'. | POST a `/api/auth/login` con contraseña 'Errada999'. | Retorna código HTTP 401 Unauthorized y ningún token es emitido. |

**16. `test_e2e_gestion_catalogo_medicamentos`** — END-TO-END — Ambos

**Objetivo:** Dar de alta un medicamento mediante POST y luego listar el catálogo con GET confirmando que el nuevo fármaco aparezca.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Token de sesión autenticado y JSON de nuevo medicamento. | POST a `/api/medicamentos` y seguido GET a `/api/medicamentos`. | POST retorna HTTP 201; GET retorna HTTP 200 con array JSON que incluye el fármaco recién creado. |

**17. `test_e2e_flujo_maestro_clinico_tratamiento_y_seguimiento_tomas`** — END-TO-END — Ambos

**Objetivo:** El gran recorrido clínico: un cuidador autenticado receta un tratamiento a su paciente, el sistema genera las tomas automáticamente y el paciente marca la primera toma como realizada.

| PREPARAR (ARRANGE) | ACTUAR (ACT) | VERIFICAR (ASSERT) |
|---|---|---|
| Cuidador y paciente vinculados, medicamento en catálogo y token JWT activo. | 1) POST `/api/tratamientos`.<br>2) GET `/api/tomas/paciente/<id>`.<br>3) PATCH `/api/tomas/<toma_id>` con `estado='tomada'`. | Tratamiento creado (201); calendario lista todas las tomas (200); PATCH confirma estado 'tomada' (200) y actualiza la base de datos. |

## 6. Cómo Ejecutar e Interpretar las Pruebas en Pareja 💻

El comando oficial que Cristian y tú deben ejecutar en la terminal para validar las pruebas es:

```
py -m unittest discover -s tests -p "test_*.py"
```

**¿Qué significan los símbolos que muestra Python en consola?**

- `.` (Punto): ¡Prueba aprobada (Verde)! La función respondió exactamente lo esperado.
- `F` (Failure): La prueba falló una comprobación (AssertionError). La lógica devolvió otro dato.

- `E` (Error): Ocurrió una excepción inesperada (código roto, variable no importada, error de sintaxis).
- `OK` al final: ¡Todas las pruebas pasaron con éxito! Criterio listo para hacer commit y merge.